



3ª Sprint do Challenge

Mastering Relational and Non-Relational Database

Professor: Gustavo Molina



Integrantes:

Gabriel Camargo – RM557879

Kauan Felipe – RM557954

Vinicius Alves – RM551939



3ª Sprint do Challenge

Introdução

1. Descrição Detalhada do Projeto

Este documento apresenta a implementação completa da Sprint 3 do projeto de banco de dados, focando no desenvolvimento de procedimentos, funções e triggers utilizando Oracle SQL. O projeto utiliza o modelo de dados da Mottu, uma empresa de logística urbana com motos elétricas, implementando funcionalidades avançadas de manipulação de dados relacionais e conversão para JSON

2. Objetivos da Solução Idealizada

- Desenvolver 2 procedimentos armazenados com lógica complexa.
- Criar 2 funções personalizadas com tratamento de exceções.
- Implementar 1 trigger de auditoria para controle de alterações.
- Aplicar boas práticas de documentação e tratamento de erros
- Converter dados relacionais para formato JSON manualmente.

3. Estrutura do Banco de Dados

3.1 Modelo Conceitual

- Filiais: Unidades da empresa espalhadas pelo país
- Pátios: Áreas físicas onde as motos ficam estacionadas
- Motos: Veículos elétricos com sensores IoT
- Câmeras: Dispositivos de monitoramento dos pátios
- Sensores IoT: Equipamentos embarcados nas motos
- Manutenções: Registros de serviços realizados
- Usuários: Técnicos responsáveis pelas manutenções



3ª Sprint do Challenge

3.2 Tabelas Principais

Tabela	Registros	Finalidade
filial	5	Unidades da empresa (SP, RJ, BH, DF, BA)
moto	5	Veículos com placas, anos e status
manutencao	5	Serviços realizados com custos
patio	5	Áreas de estacionamento com dimensões
sensor_iot	5	Dispositivos de monitoramento
camera	5	Equipamentos de segurança
usuario	5	Técnicos e suas especialidades

4. Funções Desenvolvidas

4.1 Função 1: Conversão Manual para JSON

Propósito: Converter dados relacionais em formato JSON sem usar funções nativas do Oracle.

- Realiza JOIN entre tabelas moto, patio, filial e statusmoto
- Constrói JSON manualmente através de concatenação de strings
- Trata 3 exceções específicas: TABELA_INEXISTENTE, CONSULTA_INVALIDA, DADOS_INSUFICIENTES
- Suporta filtros opcionais via parâmetro p_condicao



3ª Sprint do Challenge

```
Function CONVERTER_PARA_JSON compiled

Function VALIDAR_PLACA_MOTO compiled

=== TESTE FUNÇÃO CONVERSÃO JSON ===
[{"id_moto":1,"placa":"ABC-1234","filial":"Filial São Paulo","patio":"Pátio Principal SP","status":"Disponível","ano":"2022"}, {"id_moto":4,"placa":"JKL-3456","filial":"Filial R

=== TESTE COM FILTRO STATUS ===
[{"id_moto":1,"placa":"ABC-1234","filial":"Filial São Paulo","patio":"Pátio Principal SP","status":"Disponível","ano":"2022"}, {"id_moto":4,"placa":"JKL-3456","filial":"Filial R

PL/SQL procedure successfully completed.

TESTE 1: Parâmetro tabela NULL ou vazio
-----
] TESTE 1A: Parâmetro p_tabela = NULL
Chamada: converter_para_json(NULL)
Resultado esperado: Exceção TABELA_INEXISTENTE

Resultado obtido: {"erro": "Tabela não especificada ou inválida"}

✔ Status: SUCESSO - Exceção capturada corretamente

PL/SQL procedure successfully completed.
```

4.2 Função 2: Validação de Placa de Moto

Propósito: Validar formato e unicidade de placas de motocicletas.

Assinatura:

```
CREATE OR REPLACE FUNCTION validar_placa_moto(
    p_placa VARCHAR2
) RETURN VARCHAR2
```

Validações Implementadas:

1. Placa não nula: Verifica se o parâmetro foi informado
2. Formato correto: Usa REGEXP_LIKE para validar padrão ABC-1234
3. Unicidade: Consulta tabela moto para verificar duplicação

Tratamento de Exceções:

- PLACA_NULA: Para valores nulos ou vazios
- FORMATO_INVALIDO: Para placas que não seguem o padrão
- PLACA_DUPLICADA: Para placas já cadastradas



3ª Sprint do Challenge

```
'L/SQL procedure successfully completed.

=== TESTE FUNÇÃO VALIDAÇÃO PLACA ===
teste 1: VÁLIDA: Placa XYZ-9999 aprovada para cadastro
teste 2: INVÁLIDA: Formato deve ser ABC-1234 (3 letras, hífen, 4 números)
teste 3: INVÁLIDA: Placa já cadastrada no sistema
teste 4: INVÁLIDA: Placa não pode ser nula ou vazia

'L/SQL procedure successfully completed.
```

5. Procedimentos Desenvolvidos

5.1 Procedimento 1: Exibição de Motos em JSON

Propósito: Apresentar dados de motos em formato JSON utilizando JOINS múltiplos.

Assinatura:

```
CREATE OR REPLACE PROCEDURE exibir_motos_json(
    p_filtro_status VARCHAR2 DEFAULT NULL
)
```

Funcionalidades:

- JOIN Complexo: Une 4 tabelas (moto, patio, filial, statusmoto)
- Filtro Opcional: Permite filtrar por status específico
- Conversão JSON: Utiliza a função converter_para_json desenvolvida
- Validação: Verifica existência do status antes de aplicar filtro

Exceções Tratadas:

- STATUS_INEXISTENTE: Status informado não existe na base
- ERRO_CONVERSAO: Falha na função de conversão JSON
- NENHUM_DADO: Nenhum registro encontrado



3ª Sprint do Challenge

```
Procedure RELATORIO_CUSTOS_MANUTENCAO compiled

=== TESTE PROCEDIMENTO EXIBIR MOTOS JSON ===
=== RELATÓRIO DE MOTOS EM FORMATO JSON ===
Data/Hora: 24/09/2025 12:05:39

JSON Result:
[{"id_moto":1,"placa":"ABC-1234","filial":"Filial São Paulo","patio":"Pátio Principal SP","status":"Disponível","ano":"2022"}, {"id_moto":4,"pl

=== FIM DO RELATÓRIO ===

=== TESTE COM FILTRO DISPONÍVEL ===
=== RELATÓRIO DE MOTOS EM FORMATO JSON ===
Data/Hora: 24/09/2025 12:05:39

JSON Result:
[{"id_moto":1,"placa":"ABC-1234","filial":"Filial São Paulo","patio":"Pátio Principal SP","status":"Disponível","ano":"2022"}, {"id_moto":4,"pl

=== FIM DO RELATÓRIO ===

PL/SQL procedure successfully completed.

Teste 1.1A: Status completamente inexistente
Chamada: exibir_motos_json('StatusInexistente')
Exceção esperada: STATUS_INEXISTENTE

=== RELATÓRIO DE MOTOS EM FORMATO JSON ===
Data/Hora: 24/09/2025 12:05:39

ERRO: Status "StatusInexistente" não encontrado no sistema

PL/SQL procedure successfully completed.
```

5.2 Procedimento 2: Relatório de Custos de Manutenção

Propósito: Gerar relatório com totalizações manuais por filial e geral.

Assinatura:

CREATE OR REPLACE PROCEDURE relatorio_custos_manutencao

Lógica de Totalização Manual:

1. Carregamento: Dados armazenados em array PL/SQL
2. Ordenação: Por nome da filial e cargo do usuário
3. Subtotais: Calculados manualmente por mudança de filial
4. Total Geral: Soma acumulativa de todos os custos



3ª Sprint do Challenge

```
Registros processados: 5  
=== FIM DO RELATÓRIO ===
```

```
PL/SQL procedure successfully completed.
```

```
=== TESTE PROCEDIMENTO RELATÓRIO CUSTOS ===  
=== RELATÓRIO DE CUSTOS DE MANUTENÇÃO ===  
Data: 24/09/2025 12:09:47
```

FILIAL	CARGO	CUSTO
Filial Belo Horizonte	Supervisor	420.25
Filial Rio de Janeiro	Técnica Geral	180.00
Filial São Paulo	Especialista em Frei	225.75
Filial São Paulo	Mecânico	150.00
Filial São Paulo	Técnica Elétrica	300.50
	SUBTOTAL	1,276.50
	TOTAL GERAL	1,276.50

```
Registros processados: 5  
=== FIM DO RELATÓRIO ===
```

```
PL/SQL procedure successfully completed.
```

```
Table TEMP_BACKUP_MANUTENCAO created.
```

```
Table TEMP_BACKUP_USUARIO created.
```

```
5 rows deleted.
```

```
5 rows deleted.
```

```
Commit complete.
```

```
Teste 2.1A: Sem dados suficientes (< 5 registros)  
Chamada: relatorio_custos_manutencao()  
Exceção esperada: DADOS_INSUFICIENTES
```

```
=== RELATÓRIO DE CUSTOS DE MANUTENÇÃO ===  
Data: 24/09/2025 12:09:47
```

FILIAL	CARGO	CUSTO
ERRO: Dados insuficientes. Mínimo de 5 registros requeridos.		
Registros encontrados: 0		



3ª Sprint do Challenge

6. Trigger de Auditoria

6.1 Implementação

Trigger: tr_auditoria_moto

Tabela Monitorada: moto

Eventos: INSERT, UPDATE, DELETE

Assinatura:

```
CREATE OR REPLACE TRIGGER tr_auditoria_moto
  AFTER INSERT OR UPDATE OR DELETE ON moto
  FOR EACH ROW
```

6.2 Tabela de Auditoria

A tabela auditoria registra:

- nome_usuario: Usuário que executou a operação
- tipo_operacao: INSERT, UPDATE ou DELETE
- data_hora: Timestamp da operação
- tabela_afetada: Nome da tabela (sempre 'MOTO')
- valores_antigos: Estado anterior dos dados (:OLD)
- valores_novos: Novo estado dos dados (:NEW)

```
Trigger TR_AUDITORIA_MOTO compiled
```

```
1 row inserted.
```

```
1 row updated.
```

```
1 row deleted.
```




3ª Sprint do Challenge

All Rows Fetched: 3 in 0.024 seconds						
ID_AUDITORIA	NOME_USUARIO	TIPO_OPERACAO	DATA_HORA	TABELA_AFETADA	VALORES_ANTIGOS	VALORES_NOVOS
1	3 RM551939	DELETE	25-SEP-25 11.30.59.012427000 AM MOTO		id_moto:999 placa:TEST-456 ano_fabricacao:01/01/2025	
2	2 RM551939	UPDATE	25-SEP-25 11.30.51.013807000 AM MOTO		id_moto:999 placa:TEST-123 ano_fabricacao:01/01/2025	id_moto:999 placa:TEST-456 ano
3	1 RM551939	INSERT	25-SEP-25 11.30.42.008118000 AM MOTO			id_moto:999 placa:TEST-123 ano